



Bikkad IT Institute

Session 6: Python Functions

Compiled by Bikkad IT Institute | Teaching Notes

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Python Functions - Beginner Notes

1. Write Your First Function

What is a Function?

A function is a reusable block of code that performs a specific task.

Syntax:

Define a function

```
def greet():  
    print("Hello, Welcome to Python!")
```

Call the function

`greet()` # Output: Hello, Welcome to Python!

Example 2:

```
def add_numbers():  
    result = 5 + 10  
    print("Sum:", result)
```

`add_numbers()` # Output: Sum: 15

Key Points:

- Use `def` keyword to define a function
- Function name should be descriptive
- Use parentheses `()` after function name
- Indent the code block inside function
- Call function by writing its name with `()`

2. Parameters & Arguments

Parameters: Variables in function definition

Arguments: Actual values passed to function

'name' is a parameter

```
def greet(name):  
    print(f"Hello, {name}!")
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajnagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

'Alice' is an argument

```
greet("Alice") # Output: Hello, Alice!  
greet("Bob") # Output: Hello, Bob!
```

Multiple Parameters:

```
def add(a, b):  
    result = a + b  
    print(f"{a} + {b} = {result}")  
  
add(5, 10) # Output: 5 + 10 = 15  
add(20, 30) # Output: 20 + 30 = 50
```

3. Types of Arguments

a) Positional Arguments

Order matters - arguments assigned based on position.

```
def student_info(name, age, grade):  
    print(f"Name: {name}, Age: {age}, Grade: {grade}")  
  
student_info("Alice", 20, "A") # Output: Name: Alice, Age: 20, Grade: A
```

b) Keyword Arguments

Order doesn't matter - specify parameter names.

```
def student_info(name, age, grade):  
    print(f"Name: {name}, Age: {age}, Grade: {grade}")  
  
student_info(age=20, grade="A", name="Alice") # Output: Name: Alice, Age: 20, Grade: A
```

c) Default Arguments

Parameters with default values.

```
def greet(name, message="Welcome"):  
    print(f"{message}, {name}!")  
  
greet("Alice") # Output: Welcome, Alice!  
greet("Bob", "Good Morning") # Output: Good Morning, Bob!
```

4. *args (Arbitrary Positional Arguments)

**Use args to accept any number of positional arguments.*
It collects arguments into a **tuple**.

```
def add_all(*args):  
    print("Type:", type(args)) # Output: Type: <class 'tuple'>  
    total = sum(args)  
    return total
```

OUR LOCATIONS

```
print(add_all(1, 2, 3)) # Output: 6  
print(add_all(10, 20, 30, 40)) # Output: 100
```

**Example 2: Loop through args*

```
def print_subjects(*subjects):  
    print("Subjects:")  
    for subject in subjects:  
        print("-", subject)  
  
print_subjects("Math", "Science", "English")
```

Output:

Subjects:

- Math

- Science

- English

5. **kwargs (Arbitrary Keyword Arguments)

****Use kwargs to accept any number of keyword arguments.**
It collects arguments into a **dictionary**.

```
def student_details(**kwargs):  
    print("Type:", type(kwargs)) # Output: Type: <class 'dict'>  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
student_details(Name="Alice", Age=20, City="Mumbai")
```

Output:

Type: <class 'dict'>

Name: Alice

Age: 20

City: Mumbai

****Combining *args and kwargs:**

```
def display_info(*args, **kwargs):  
    print("Positional arguments:", args)  
    print("Keyword arguments:", kwargs)  
  
display_info(1, 2, 3, name="Alice", age=20)
```

Output:

Positional arguments: (1, 2, 3)

Keyword arguments: {'name': 'Alice', 'age': 20}

Order Rule: Regular parameters → *args → **kwargs

6. How Functions Are Executed in Memory

Function execution follows these steps:

1. **Function Call:** Program encounters function call
2. **Memory Allocation:** New stack frame created in memory
3. **Parameter Binding:** Arguments assigned to parameters
4. **Code Execution:** Function body executes line by line
5. **Return:** Function returns value (if any)
6. **Memory Cleanup:** Stack frame removed from memory

```
def multiply(a, b):  
    result = a * b # Local variable created in memory  
    return result # Value returned  
  
answer = multiply(5, 10) # Function called, memory allocated  
print(answer) # Output: 50
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

After execution, function's local variables are cleared

Key Point: Variables inside functions are **local** (exist only during execution).

7. Without Return Statement

If no return statement, function returns None by default.

```
def greet(name):  
    print(f"Hello, {name}!")  
    # No return statement  
  
result = greet("Alice") # Output: Hello, Alice!  
print(result) # Output: None
```

With Return:

```
def add(a, b):  
    return a + b  
  
result = add(5, 10)  
print(result) # Output: 15
```

Return Multiple Values:

```
def calculate(a, b):  
    addition = a + b  
    multiplication = a * b  
    return addition, multiplication # Returns tuple  
  
result = calculate(5, 10)  
print(result) # Output: (15, 50)
```

8. Variable Scope

Scope determines where a variable can be accessed.

a) Local Scope

Variables defined inside a function.

```
def my_function():  
    x = 10 # Local variable  
    print(x)  
  
my_function() # Output: 10
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

print(x) # Error: x is not defined outside function

b) Global Scope

Variables defined outside all functions.

```
x = 100 # Global variable
```

```
def my_function():  
    print(x) # Can access global variable
```

```
my_function() # Output: 100  
print(x) # Output: 100
```

c) Modifying Global Variable

```
x = 100
```

```
def my_function():  
    global x # Declare x as global  
    x = 200 # Modify global variable
```

```
my_function()  
print(x) # Output: 200
```

9. Nested Functions

Function defined inside another function.

```
def outer():  
    print("Outer function")
```

```
def inner():  
    print("Inner function")  
  
inner() # Call inner function
```

```
outer()
```

Output:

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Outer function

Inner function

Accessing Outer Variable:

```
def outer():  
    x = "Hello"
```

```
    def inner():  
        print(x) # Access outer function's variable  
  
    inner()
```

outer() # Output: Hello

Using nonlocal:

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x # Modify outer function's variable  
        x = 20  
  
    inner()  
    print(x)
```

outer() # Output: 20

10. Functions Are First-Class Citizens

Functions can be:

- Assigned to variables
- Passed as arguments
- Returned from other functions

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

a) Assign Function to Variable

```
def greet():  
    return "Hello!"
```

```
say_hello = greet # Assign function to variable  
print(say_hello()) # Output: Hello!
```

b) Pass Function as Argument

```
def execute(func):  
    print("Executing function:")  
    func()
```

```
def greet():  
    print("Hello, World!")
```

```
execute(greet)
```

Output:

Executing function:

Hello, World!

c) Return Function from Function (covered in next section)

11. Deletion of a Function

Use **del** to delete a function.

```
def greet():  
    print("Hello!")
```

```
greet() # Output: Hello!
```

```
del greet # Delete the function
```

greet() # Error: name 'greet' is not defined

12. Returning a Function

Functions can return other functions.

```
def outer():  
    def inner():  
        return "Hello from inner function!"  
    return inner # Return function
```

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

```
my_func = outer()
print(my_func()) # Output: Hello from inner function!
```

Practical Example:

```
def multiplier(n):
    def multiply(x):
        return x * n
    return multiply

times_2 = multiplier(2)
times_5 = multiplier(5)

print(times_2(10)) # Output: 20
print(times_5(10)) # Output: 50
```

13. Benefits of Using Functions

- **Code Reusability** - Write once, use multiple times
- **Organization** - Break complex problems into smaller parts
- **Maintainability** - Easy to update and debug
- **Readability** - Makes code easier to understand
- **Modularity** - Independent components
- **Testing** - Easy to test individual functions

Example:

Without function (repetitive)

```
print(5 * 5)
print(10 * 10)
print(15 * 15)
```

With function (reusable)

```
def square(n):
    return n * n

print(square(5))
print(square(10))
print(square(15))
```

14. Lambda Functions

Lambda is an anonymous (unnamed) function with a single expression.

Syntax: lambda arguments: expression

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Normal function

```
def square(x):  
    return x ** 2
```

Lambda function

```
square_lambda = lambda x: x ** 2  
  
print(square(5)) # Output: 25  
print(square_lambda(5)) # Output: 25
```

Multiple Arguments:

Addition

```
add = lambda a, b: a + b  
print(add(5, 10)) # Output: 15
```

Multiplication

```
multiply = lambda x, y, z: x * y * z  
print(multiply(2, 3, 4)) # Output: 24
```

Conditional Lambda:

Check if number is greater than 10

```
check = lambda x: x if x > 10 else 10  
print(check(15)) # Output: 15  
print(check(5)) # Output: 10
```

15. Difference Between Lambda vs Normal Functions

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Feature	Lambda Function	Normal Function (def)
Definition	lambda x: x * 2	def func(x): return x * 2
Name	Anonymous (no name)	Must have a name
Expressions	Single expression only	Multiple statements allowed
Return	Implicit return	Explicit return keyword
Readability	Less readable for complex logic	More readable
Use Case	Short, temporary functions	Reusable, complex functions
Documentation	No docstring support	Can have docstrings
Lines	Single line only	Multiple lines allowed

Table 1: Lambda vs Normal Functions

Example Comparison:

Lambda

```
square = lambda x: x ** 2
```

Normal Function

```
def square_func(x):
    """Returns square of a number"""
    return x ** 2
```

```
print(square(5)) # Output: 25
print(square_func(5)) # Output: 25
```

16. Higher-Order Functions

Functions that:

- Take other functions as arguments, OR
- Return functions as results

OUR LOCATIONS

+91 77220 82225
Pimpri-Chinchwad

+91 77220 82226
Chh. Sambhajinagar

+91 77220 82227
Nashik

+91 77220 82224
Hadapsar, Pune

Function that takes another function as argument

```
def apply_operation(func, value):  
    return func(value)  
  
def square(x):  
    return x ** 2  
  
result = apply_operation(square, 5)  
print(result) # Output: 25
```

With lambda

```
result2 = apply_operation(lambda x: x * 3, 5)  
print(result2) # Output: 15
```

17. Map

map() applies a function to every item in an iterable.

Syntax: map(function, iterable)

Example 1: Square all numbers

```
numbers = [1, 2, 3, 4, 5]  
squared = map(lambda x: x ** 2, numbers)  
print(list(squared)) # Output: [1, 4, 9, 16, 25]
```

With Normal Function:

```
def double(x):  
    return x * 2  
  
numbers = [1, 2, 3, 4, 5]  
result = map(double, numbers)  
print(list(result)) # Output: [2, 4, 6, 8, 10]
```

Multiple Iterables:

```
a = [1, 2, 3]  
b = [4, 5, 6]  
result = map(lambda x, y: x + y, a, b)  
print(list(result)) # Output: [5, 7, 9]
```

OUR LOCATIONS

18. Filter

filter() filters items based on a condition (returns only True items).

Syntax: filter(function, iterable)

Example 1: Filter even numbers

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even = filter(lambda x: x % 2 == 0, numbers)
print(list(even)) # Output: [2, 4, 6, 8, 10]
```

Filter Strings:

```
fruits = ["Apple", "Banana", "Apricot", "Cherry", "Avocado"]
a_fruits = filter(lambda fruit: fruit.startswith("A"), fruits)
print(list(a_fruits)) # Output: ['Apple', 'Apricot', 'Avocado']
```

Filter Numbers Greater Than 5:

```
numbers = [3, 8, 1, 6, 12, 4, 9]
result = filter(lambda x: x > 5, numbers)
print(list(result)) # Output: [8, 6, 12, 9]
```

19. Reduce

reduce() applies a function cumulatively to items, reducing to a single value.

Syntax: reduce(function, iterable, initial_value)

Import: from functools import reduce

from functools import reduce

Example 1: Sum of all numbers

```
numbers = [1, 2, 3, 4, 5]
total = reduce(lambda x, y: x + y, numbers)
print(total) # Output: 15
```

How it works:

1. reduce calls lambda(1, 2) → returns 3
2. reduce calls lambda(3, 3) → returns 6
3. reduce calls lambda(6, 4) → returns 10
4. reduce calls lambda(10, 5) → returns 15

Product of Numbers:

from functools import reduce

OUR LOCATIONS

```
numbers = [2, 3, 4]
product = reduce(lambda x, y: x * y, numbers)
print(product) # Output: 24
```

With Initial Value:

```
from functools import reduce

numbers = [1, 2, 3, 4]
result = reduce(lambda x, y: x + y, numbers, 10)
print(result) # Output: 20 (10 + 1 + 2 + 3 + 4)
```

Find Maximum:

```
from functools import reduce

numbers = [5, 12, 8, 20, 3]
maximum = reduce(lambda x, y: x if x > y else y, numbers)
print(maximum) # Output: 20
```

Quick Reference

Concept	Example
Define function	def greet(): print("Hi")
Call function	greet()
Parameters	def add(a, b): return a + b
*args	def func(*args): print(args)
kwargs	def func(kwargs): print(kwargs)
Lambda	lambda x: x * 2
Map	map(lambda x: x**2, [1,2,3])
Filter	filter(lambda x: x>5, [1,6,3,8])
Reduce	reduce(lambda x,y: x+y, [1,2,3])

Table 2: Functions Quick Reference

Function	Purpose
map()	Transform each item
filter()	Select items based on condition
reduce()	Combine all items to single value

Table 3: Map, Filter, Reduce Comparison

OUR LOCATIONS